

CubeFit

Architecture, mathematics, operation, diagnostics, and validation

Documentation revision: 3 October 2026

Grounded in the supplied complete Repomix repository snapshot.

Current production pathway

```
genCubeFit -> PipelineRunner.solve_all_mp_batched  
-> solve_streaming_nnls -> streamActiveSetNNLS
```

Contents

Contents	2
1. Purpose and scope	5
2. Core notation	5
2.1 Array orientation	6
3. Scientific meaning of the HyperCube	6
3.1 Dynamical information lives in A	6
3.2 What $X[c,p]$ controls	6
4. High-level workflow: <code>genCubeFit()</code>	7
4.1 Important <code>**kwargs</code>	7
4.2 <code>runSwitch</code>	7
4.3 Output names	7
5. HDF5 backbone	8
5.1 Core datasets	8
5.2 Component weights	8
5.3 Data-flux vector	8
5.4 Template guard and cropping	8
6. HDF5 opening, cache, and locking	8
7. HyperCube preflight	9
8. HyperCube construction	9
8.1 Build sequence	9
8.2 Main output	10
8.3 Resumability	10
8.4 Component support	10
9. HyperCube normalization	10
9.1 Model normalization	10
9.2 Data normalization	10
9.3 Post-hoc conversion	10
10. HyperCube multiprocessing and BLAS threads	11
10.1 Important implementation detail in this snapshot	11
11. Hybrid parallelism resolution	11
11.1 Recommended launch environment	11
12. PipelineRunner	12
13. Warm starts and resume	12
13.1 Explicit x0	12
13.2 <code>warm_start='zeros'</code>	12
13.3 <code>warm_start='saved_x'</code>	12
13.4 <code>warm_start='resume'</code>	12
14. Full-resume compatibility	12

15. Production streaming solver	13
15.1 First streaming pass	13
15.2 Fused cache	13
15.3 Current lambda-weight behavior	13
16. Exact column-energy scaling	13
17. Known-zero columns	14
18. Objective and scientific regularization	14
18.1 Numerical ridge is separate	14
19. The v2.0 orbit constraint	15
19.1 Canonicalization	15
19.2 What the weights mean now	15
19.3 Constraint equation	15
19.4 Why the old proportional form was wrong here	15
19.5 Interpretation of alpha	15
20. Reduced constrained KKT solve	15
21. Active support	16
22. KKT diagnostics	16
23. Joint-support exploration	17
23.1 Exchange screening	17
23.2 Data-gradient screening	17
23.3 Rotating coverage	17
24. Coarse-to-fine dyadic exploration	17
24.1 Round-robin batch assembly	17
25. Exploration acceptance	18
26. Checkpointing and FitTracker	18
27. Solver diagnostics and JSONL	18
27.1 Setup records	19
27.2 Plotting diagnostics	19
28. Environment variables: current versus historical	19
28.1 Live in the current production path	19
28.2 Historical or auxiliary for the current constrained solve	19
29. Reconstruction of /ModelCube	20
30. loadCubeFit() diagnostics	20
31. Orbit-constraint plot	20
32. Validation and exact closure tests	21
32.1 Create a minimal mock-data file	21
32.2 Load validation data in genCubeFit()	21
32.3 Validation output separation	21
32.4 Compare recovered coefficients with truth	21
32.5 What a closure failure means	22

33. Storage management	22
33.1 Archive and restore	22
34. Current solution format	22
35. Debugging utilities	23
36. Reference and audit solvers	23
37. Performance interpretation	23
37.1 HyperCube build	23
37.2 Solver	23
37.3 Sparse support	23
38. Troubleshooting	24
38.1 ... does not contain /ModelCube	24
38.2 Validation output file does not exist	24
38.3 Mock-data file is tens of GB	24
38.4 Constrained fit is much worse than unconstrained fit	24
38.5 Full resume rejected	24
38.6 More CPUs are slower	24
38.7 Orbit plot target looks wrong	24
38.8 /X_global shape assumptions fail	25
39. Recommended production checklist	25
40. Source-derived API index	25
40.1 kz_fitSpec.py	25
40.2 hypercube_builder.py	26
40.3 hdf5_manager.py	26
40.4 pipeline_runner.py	26
40.5 streaming_nnls_constrained.py	26
40.6 validation.py	26
40.7 plotting.py	26
40.8 cube_utils.py	26
40.9 cube_debug.py	27
41. Building the repository documentation	27
42. Snapshot-specific audit notes	27
43. Maintenance rule	27

1. Purpose and scope

CubeFit is an HDF5-backed spectral-fitting pipeline for assigning stellar populations to dynamical components of a spatially resolved galaxy model. Its central design goal is to fit the observed IFU spectra without discarding the spatial and line-of-sight velocity structure supplied by the dynamical model.

The current repository contains several generations of solver code. This manual documents the pathway actually selected by the high-level `kz_fitSpec.genCubeFit()` function in the supplied snapshot:

```
kz_fitSpec.genCubeFit
|
+--> H5Manager.populate_from_arrays
|
|      +--> /DataCube, /LOSVD, /Templates, grids, /R_T
|
+--> assert_preflight_ok
|
+--> estimate_global_velocity_bias_prebuild [optional]
|
+--> PipelineRunner.build_hypercube
|
|      +--> hypercube_builder.build_hypercube
|
|          |
|          +--> /HyperCube/models
|          +--> /HyperCube/_done
|          +--> /HyperCube/col_energy
|
+--> PipelineRunner.solve_all_mp_batched
|
|      +--> streaming_nnls_constrained.solve_streaming_nnls
|
|          |
|          +--> streamed A^T y and column energies
|          +--> exact column scaling
|          +--> streamActiveSetNNLS
|
|              |
|              +--> joint reduced active-set solves
|              +--> optional hard orbit constraint
|              +--> KKT checks
|              +--> joint-support exploration
|              +--> checkpoint state
```

The old documentation described a multiprocess Kaczmarz solver with trust regions, backtracking, lambda-weighted updates, and a tile-local NNLS polish. Those descriptions are historical for the current production path. The current `PipelineRunner.solve_all_mp_batched()` calls `solve_streaming_nnls()` from `streaming_nnls_constrained.py`.

2. Core notation

The following notation is used throughout the code and this manual.

Symbol	Meaning	Principal HDF5 object
S	spatial bins / apertures	<code>/DataCube.shape[0]</code>
L	observed wavelength pixels	<code>/DataCube.shape[1]</code>
T	cropped template-grid pixels	<code>/Templates.shape[1]</code>
V	LOSVD velocity bins	<code>/LOSVD.shape[1]</code>
C	dynamical/orbit components	<code>/LOSVD.shape[2]</code>
P	flattened SSP populations	<code>/Templates.shape[0]</code>
CP	total coefficients	<code>C * P</code>
<code>Y[s, l]</code>	observed spectrum	<code>/DataCube</code>

Symbol	Meaning	Principal HDF5 object
$H[s, v, c]$	LOSVD histogram	/LOSVD
$T[p, t]$	SSP spectrum	/Templates
$A[s, c, p, l]$	HyperCube basis	/HyperCube/models
$X[c, p]$	physical fit coefficient	/X_global

The model is

$$Y_{\text{hat}}[s, l] = \sum_c \sum_p A[s, c, p, l] X[c, p].$$

X is global: the same coefficient for a given (c, p) applies in every spatial bin. The spatial dependence enters through A , which contains the LOSVD-convolved dynamical component in every bin.

2.1 Array orientation

The high-level spectral preparation code carries the observed data internally as wavelength-major (L, S) . `H5Manager.populate_from_arrays()` explicitly expects this orientation and stores the data as (S, L) in `/DataCube`.

The LOSVD is stored as (S, V, C) .

The SSP library supplied to `populate_from_arrays()` has spectral axis first:

```
(T, population_axis_1, population_axis_2, ...)
```

The manager moves the spectral axis to the end and flattens the population axes to (P, T) . Metadata on `/Templates` records the original shape and crop indices so the population axes can be reconstructed later.

3. Scientific meaning of the HyperCube

3.1 Dynamical information lives in A

For each spatial bin and dynamical component, the input LOSVD carries both a velocity distribution and an amplitude. The HyperCube builder separates these conceptually into a unit-area convolution kernel and a scalar amplitude.

With the production setting

```
norm_mode='model'
```

the scalar LOSVD amplitude is multiplied into every population column of that component:

```
A[s, c, p, :] = LOSVD-convolved/rebinned SSP[p, :] * amplitude[s, c].
```

This is a central scientific invariant of the current model. The relative strength of the dynamical components is already present in the design matrix.

3.2 What $X[c, p]$ controls

$X[c, p]$ controls the stellar-population mixture assigned to component c . Without an additional constraint, NNLS is also free to change the total coefficient mass

```
m_c = sum_p X[c, p].
```

That extra degree of freedom can rescale one dynamical component relative to another even though the input HyperCube already contains the intended Schwarzschild component amplitudes.

The current hard orbit constraint removes that component-level rescaling while leaving the population distribution within each component free.

4. High-level workflow: `genCubeFit()`

The public high-level generator is

```
genCubeFit(galaxy, mPath, decDir=None, nCuts=None, proj='i', SN=90,
            full=False, slope=1.30, IMF='KB', iso='pad', weighting='luminosity',
            lOrder=4, rescale=False, specRange=None, lsf=False, band='r', smask=None,
            method='fsf', varIMF=False, source='ppxf', redraw=False,
            runSwitch='gen', bias=True, **kwargs)
```

The function performs substantially more than the name suggests. It reads the dynamical decomposition, loads or generates aperture masses and LOSVD histograms, prepares the spectral data and SSP grid through `_oneTimeSpec`, constructs the wavelength mask, resolves hybrid parallelism, creates the HDF5 file, preflights the convolution, builds the HyperCube, and optionally solves for `x`.

4.1 Important `**kwargs`

The current function removes these values from `kwargs` explicitly:

Keyword	Current meaning
<code>warm</code>	warm-start mode; default <code>zeros</code>
<code>regularisation_scale</code>	scientific L2 strength; default <code>1.0</code>
<code>validationPath</code>	synthetic validation input HDF5
<code>validationDataset</code>	synthetic spectra dataset; default <code>/DataCube</code>
<code>validationTag</code>	required output tag for validation runs
<code>cpu_processes</code>	requested worker processes; default module constant <code>12</code>
<code>blas_threads</code>	requested BLAS threads/process; default module constant <code>4</code>
<code>hdf5Dir</code>	HDF5 output directory
<code>orbitWeights</code>	if false, pass <code>None</code> to solver; default <code>false</code>

`regularisation_scale` must be finite and non-negative.

4.2 `runSwitch`

The current control flow is substring based.

- If `'gen'` in `runSwitch`, `CubeFit` builds/ensures the HyperCube and returns.
- Otherwise, the function requires `'fit'` in `runSwitch` before solving.

Therefore `runSwitch='fit'` is the normal build-if-needed-and-fit mode. A string containing `gen` returns before fitting.

4.3 Output names

The HyperCube filename is constructed as

```
hypercube_<C>_<lOrder>.h5
```

or, for a validation run,

```
hypercube_<C>_<lOrder>_<validationTag>.h5.
```

The sibling solution filename replaces the first `hypercube` with `x`.

The solver also writes `/X_global` into the main HyperCube file through `PipelineRunner`; `genCubeFit()` then writes the same final solution into the sibling `x_*.h5` product.

5. HDF5 backbone

`H5Manager.populate_from_arrays()` is the main writer used by `genCubeFit()`. It validates the array shapes, crops the template wavelength range to the observed grid plus a velocity guard, builds the rebin operator, and writes the canonical arrays.

5.1 Core datasets

Dataset	Shape	Type	Meaning
<code>/DataCube</code>	(S, L)	float64	observed or validation spectra
<code>/LOSVD</code>	(S, V, C)	float64	LOSVD histograms
<code>/Templates</code>	(P, T)	float64	cropped, flattened SSP spectra
<code>/TemPix</code>	$(T,)$	float64	template log-wavelength grid
<code>/ObsPix</code>	$(L,)$	float64	observed log-wavelength grid
<code>/VelPix</code>	$(V,)$	float64	velocity grid
<code>/R_T</code>	(T, L)	float32/64	template-to-observed rebin operator
<code>/RebinMatrix</code>	(L, T)	float32/64	transpose mapping
<code>/Mask</code>	$(L,)$	bool	true means wavelength is fitted

Optional spatial metadata are stored as `/XPix`, `/YPix`, `/BinNum`, and `/BinCounts` when supplied.

5.2 Component weights

If `orbit_weights` is supplied to `populate_from_arrays()`, the current code reduces a possible $(C \times P,)$ input to $(C,)$, unit-sum normalizes it, and stores it as

```
/CompWeights
```

with metadata describing it as the prior component weights.

The implementation writes `/CompWeights`; an older function docstring still mentions `/OrbitWeights`, which is stale.

5.3 Data-flux vector

`populate_from_arrays()` computes

```
/HyperCube/data_flux (S, )
```

as the mean `/DataCube` flux over wavelengths retained by `/Mask`. This is used by `norm_mode='data'` in the HyperCube builder.

5.4 Template guard and cropping

The manager derives the maximum velocity-induced pixel shift from `/VelPix` and the template log-wavelength spacing. It adds `safety_pad_px` and crops the template grid to the smallest region that still covers the observed grid plus the guard.

If the original template grid cannot supply the requested guard, the manager raises rather than silently convolving with insufficient support.

6. HDF5 opening, cache, and locking

All principal code paths use `hdf5_manager.open_h5()`.

By default it:

- opens readers with mode `r` and writers with mode `a`;
- uses `libver='latest'`;
- defaults `HDF5_USE_FILE_LOCKING` to `FALSE` if the environment did not already set it;
- retries likely lock/consistency failures; and
- configures the HDF5 raw chunk cache.

Current cache environment variables are:

Variable	Default in <code>open_h5()</code>
<code>CUBEFIT_RDCC_NBYTES</code>	<code>4 * 1024**3 bytes</code>
<code>CUBEFIT_RDCC_NSLOTS</code>	<code>400003</code>
<code>CUBEFIT_RDCC_W0</code>	<code>0.9</code>

These are live in the current HDF5 pathway.

7. HyperCube preflight

Before the expensive build, `genCubeFit()` calls `assert_preflight_ok()` on a small selection of spatial bins, components, and populations.

The preflight checks the LOSVD convolution and rebin pathway against direct references, including relative error, pixel shift, flat-response behavior, and the rebin operator. The high-level call currently uses tolerances

```
tol_rel      = 2e-3
tol_shift_px = 0.5
tol_flat_valid = 3e-8
rt_flat_tol  = 3e-8
```

If `bias=True`, `estimate_global_velocity_bias_prebuild()` is then used to estimate a global velocity offset before HyperCube construction. The resulting velocity bias is passed into the builder.

8. HyperCube construction

The builder signature is

```
build_hypercube(base_h5, *, norm_mode='data', amp_mode='sum',
               cp_flux_ref_mode='median', S_chunk=128, C_chunk=1, P_chunk=360,
               compression=None, vel_bias_kms=0.0, processes=None,
               blas_threads=None)
```

`genCubeFit()` currently calls it through `PipelineRunner` with

```
norm_mode = model
amp_mode  = sum
S_chunk   = 128
C_chunk    = 2
P_chunk    = 360
```

plus the resolved process and BLAS settings.

8.1 Build sequence

For every (`s-tile`, `c-tile`, `p-block`) slab, the builder:

1. reads the native LOSVDs for the spatial/component tile;
2. maps each LOSVD to a unit convolution kernel on the template grid;
3. convolves the selected SSP block using FFTs;
4. crops the linear convolution to the template length;
5. multiplies by `/R_T` to reach the observed wavelength grid;

6. clips negative numerical values to zero;
7. applies the selected scalar normalization once;
8. accumulates column-energy diagnostics; and
9. writes the result to `/HyperCube/models`.

8.2 Main output

```
/HyperCube/models      (S,C,P,L) float32
```

The dataset chunks are

```
(min(S_chunk,S), min(C_chunk,C), min(P_chunk,P), L) .
```

The last wavelength dimension is therefore stored whole within each chunk.

8.3 Resumability

The builder maintains `/HyperCube/_done`, indexed over the (S, C, P, L) tile grid. A completed file is marked with

```
/HyperCube.attrs['complete'] = True.
```

If the group is already complete, the bitmap is entirely done, and the model shape matches (S, C, P, L) , `build_hypercube()` exits without opening a writer.

`invalidate_done()` can be used to force regeneration.

8.4 Component support

The builder writes `/HyperCube/component_support`, a tile-by-component uint8 mask. A component is marked active in a spatial tile if any LOSVD amplitude in that tile exceeds a small threshold relative to the component maximum.

The builder also accumulates tile/component energy internally while building.

9. HyperCube normalization

9.1 Model normalization

For `norm_mode='model'`, the builder uses

```
scale[s,c] = losvd_amp[s,c].
```

This is the production path selected by `genCubeFit()`.

The consequence is scientifically important: the relative dynamical strength of component c in spatial bin s is already embedded in every `A[s, c, p, :]` column.

9.2 Data normalization

For `norm_mode='data'`, the scale is

```
scale[s,c] = data_flux[s] * losvd_amp[s,c] / sum_c losvd_amp[s,c]
```

when the denominator and data flux are positive.

This preserves the LOSVD component fractions within a spatial bin but ties the sum to the observed per-bin mean flux.

9.3 Post-hoc conversion

`convert_hypercube_norm()` can convert an existing HyperCube between model and data normalization without repeating the FFT convolution. It rescales the stored models spatially and can recompute column energies.

Do not interpret a fitted x without checking the HyperCube normalization. The physical meaning of component-level coefficient mass depends on where the dynamical amplitude is encoded.

10. HyperCube multiprocessing and BLAS threads

The current builder accepts both `processes` and `blas_threads`. If omitted, it reads

```
CUBEFIT_GEN_PROCESSES
CUBEFIT_GEN_BLAS_THREADS
```

with a default of one for each.

Workers are created with the multiprocessing `spawn` context. Their initializer sets

```
OMP_NUM_THREADS
OPENBLAS_NUM_THREADS
MKL_NUM_THREADS
```

to the requested BLAS-thread count.

It does not set `BLIS_NUM_THREADS`. If NumPy/SciPy is linked to BLIS, set `BLIS_NUM_THREADS` before Python starts.

10.1 Important implementation detail in this snapshot

Although `build_hypercube()` creates a `ProcessPoolExecutor` with `max_workers=processes`, the supplied implementation submits one slab future, waits for that future through `as_completed(jobs)`, writes the result, and only then advances to the next tile.

Therefore this exact snapshot does not keep several HyperCube tile jobs in flight simultaneously. A value `processes>1` creates a larger worker pool, but only one worker receives a tile at a time. This is relevant when interpreting wall-time scaling.

The work unit itself is nevertheless the newer coarse (`s-tile`, `c-tile`, `p-block`) slab rather than the older fine-grained per-cell job.

11. Hybrid parallelism resolution

`genCubeFit()` obtains

```
best_processes, best_blas = resolve_parallelism(
    requested_processes, requested_blas)
```

before constructing the HyperCube or solver pool.

`resolve_parallelism()` uses the process CPU affinity/cpuset as the available core count. If the requested product fits, it preserves the request. Otherwise it searches feasible process/thread pairs and prefers configurations that:

1. avoid oversubscription;
2. use as much of the allocation as possible;
3. retain both process and BLAS parallelism where possible; and
4. remain similar to the requested process-to-thread balance.

This makes the Slurm cpuset, not the nominal node CPU count, authoritative.

11.1 Recommended launch environment

Set BLAS/OpenMP controls in the Slurm script or shell before Python imports NumPy:

```
export OMP_NUM_THREADS=2
export OPENBLAS_NUM_THREADS=2
export MKL_NUM_THREADS=2
export BLIS_NUM_THREADS=2
export NUMEXPR_NUM_THREADS=1
export PYTHONUNBUFFERED=1
```

The exact numbers should be consistent with the Slurm allocation and the requested CubeFit process count.

12. PipelineRunner

`PipelineRunner(h5_path)` reads dimensions from root metadata when available and otherwise infers them from `/DataCube`, `/LOSVD`, and `/Templates`.

The main production solve entry point is

```
runner.solve_all_mp_batched(
    reader_s_tile=128,
    reader_c_tile=1,
    reader_p_tile=360,
    reader_dtype_models='float32',
    reader_apply_mask=True,
    processes=2,
    blas_threads=12,
    orbit_weights=None,
    x0=None,
    warm_start='zeros',
    regularisation_scale=1.0,
    tracker_mode='on',
)
```

Despite the historical method name, this function no longer runs the old batched Kaczmarz algorithm. It creates an `MPConfig` and calls `streaming_nnls_constrained.solve_streaming_nnls()`.

13. Warm starts and resume

The runner implements three named warm-start modes plus an explicit `x0`.

13.1 Explicit `x0`

If `x0` is passed, it is validated, flattened, and used as the physical solution seed with fresh solver state.

13.2 `warm_start='zeros'`

The runner creates a zero vector of length `C*P`.

13.3 `warm_start='saved_x'`

The runner searches the newest valid physical solution from:

- the latest `FitTracker` sidecar `/Fit/x_last`; and
- the main HDF5 `/X_global`.

It chooses the newer valid source by file modification time. Only the physical solution is reused; active-set/search state is fresh.

This is the safest warm start after changing solver mathematics.

13.4 `warm_start='resume'`

The runner locates the latest sidecar and attempts `load_checkpoint()`. A full checkpoint requires both `/Fit/x_last` and a matching serialized solver state with the same iteration number.

If full state is unavailable, the runner falls back to `saved_x` behavior.

14. Full-resume compatibility

`streamActiveSetNNLS()` validates checkpoint metadata before using it.

A full resume is rejected when the requested `regularisation_scale` differs from the saved value.

The solver also checks constrained versus unconstrained mode. Historical checkpoints without a `hard_orbit_shape` flag are interpreted as constrained, which preserves compatibility with older constrained checkpoints rather than silently treating them as unconstrained.

For a current constrained checkpoint, the saved

```
orbit_constraint_mode
```

must equal

```
equal_component_mass.
```

If the checkpoint contains `orbit_shape`, it must also match the currently supplied canonical orbit weights.

After changing orbit weights, regularization, or the orbit-constraint mathematics, use `warm_start='saved_x'` rather than full `resume`.

15. Production streaming solver

The production function is

```
solve_streaming_nnls(h5_path, cfg, *, orbit_weights=None, x0=None,
    resume_state=None, tracker=None, block_size=None,
    monolithic_max_active=1000, regularisation_scale=1.0,
    cache_path=None, reuse_cache=True)
```

Its current docstring still refers to a fused block-coordinate solver, but the actual production flow in this snapshot performs a streaming first pass and then calls the monolithic streaming active-set solver

```
streamActiveSetNNLS().
```

15.1 First streaming pass

The solver reads `/DataCube` and `/HyperCube/models` spatial tile by spatial tile. It computes:

```
ATy_flat[j] = A_j^T y
D_tot[c,p] = sum A[s,c,p,l]^2
```

over the selected wavelength mask.

It does not materialize the complete dense design matrix in RAM.

15.2 Fused cache

The first-pass products can be stored in

```
<h5_path>.bcfused.npz
```

or a user-specified `cache_path`.

Cache reuse checks `S`, `L`, `C`, `P`, spatial tile size, mask application, and the exact retained wavelength indices. A structurally incompatible cache is closed and rebuilt.

15.3 Current lambda-weight behavior

The current production first pass applies `/Mask` but does not read or apply `/HyperCube/lambda_weights` to `ATy_flat` or `D_tot`.

This differs from the old Kaczmarz documentation. Lambda-weight utilities and historical lambda-related environment variables remain in the repository, but they are not part of the current `solve_streaming_nnls()` objective.

16. Exact column-energy scaling

After the first pass, the solver converts summed energy to average energy per spatial tile:

```
E[c,p] = D_tot[c,p] / n_tiles.
```

It then constructs

```
S[c,p] = 1 / sqrt(E[c,p])
```

for positive-energy columns.

There is deliberately no median floor or cap in the current code. A column with non-positive energy must already be structurally excluded by the `known_zero_mask`; otherwise the solver raises.

The active solver works in scaled coordinates z , while physical coefficients satisfy

```
x = S * z.
```

This scaling is conditioning machinery. The final returned and stored `X_global` is in physical coefficient space.

17. Known-zero columns

The solver reads the persistent mask

```
/HyperCube/known_zero_mask (C,P)
```

when present. `True` entries are hard exclusions.

If no mask exists, the solver starts with all entries allowed.

In constrained mode, a positive-weight component cannot have every population marked known-zero; such a configuration makes the hard component constraint infeasible and raises immediately.

A column that simply failed an exploration trial is not the same thing as a structurally known-zero column. The persistent mask should represent genuine hard exclusions.

18. Objective and scientific regularization

The active solver minimizes a non-negative quadratic data objective plus an optional physical-space L2 term.

Conceptually,

```
minimize 0.5 ||A x - y||^2
          + 0.5 * regularisation_scale * ||x||^2
subject to x >= 0
```

with the orbit equality constraints added when enabled.

Because the solver works in z coordinates with $x = S * z$, the scientific regularization contributes

```
regularisation_scale * diag(S^2)
```

to the reduced z -space Hessian.

18.1 Numerical ridge is separate

The active solver also computes a small adaptive numerical ridge to control ill-conditioning of reduced systems. The code explicitly separates this from `regularisation_scale`.

The numerical ridge is chosen relative to a target condition number and is reported separately in diagnostics as `max_grad_numerical_ridge / ridge`.

Set

```
regularisation_scale=0.0
```

for a strict unregularized validation/reference fit. The solver may still use a tiny numerical ridge for stable reduced linear algebra.

19. The v2.0 orbit constraint

This is the most important mathematical change relative to the old documentation.

19.1 Canonicalization

`_canon_orbit_weights()` accepts:

- `None`;
- one value per component $(C,)$; or
- one value per coefficient $(C * P,)$, which is summed to component level.

Weights must be finite, non-negative, and have positive total mass. `Non-None` weights are unit-sum normalized.

19.2 What the weights mean now

Under the production model-normalized HyperCube, the relative dynamical amplitudes are already encoded in A .

Therefore the normalized orbit-weight values are not coefficient-mass target fractions.

They select the components participating in the hard constraint:

```
positive weight  -> constrained component
zero weight      -> excluded from the equal-mass constraint/support
```

19.3 Constraint equation

For every positive-weight component,

```
sum_p X[c,p] = alpha,
```

where the single $\alpha \geq 0$ is fitted jointly with the active coefficients.

The solver reports this mode as

```
orbit_constraint_mode = equal_component_mass.
```

19.4 Why the old proportional form was wrong here

The older constrained formulation used the orbit shape as a target in coefficient space, effectively requiring a form like

```
sum_p X[c,p] proportional to w_c.
```

For `norm_mode='model'`, this applies the component-weight ratios twice: first through the HyperCube amplitude and again through X .

The v2.0 formulation instead uses the HyperCube to preserve the dynamical mass ratios and constrains only the otherwise arbitrary per-component coefficient normalization.

19.5 Interpretation of α

In the current code, α is the common coefficient mass of every participating component. It is fitted from the spectral data together with the active population coefficients.

It can be understood as the global coefficient normalization connecting the model-normalized Schwarzschild basis to the observed spectral scale.

20. Reduced constrained KKT solve

`_orbit_hard_constraint_solve()` solves the active reduced problem in scaled coordinates.

For active coefficient vector z , the physical component mass is

```
m_c = sum_{j in component c} S_j z_j.
```

The KKT unknowns are ordered as

```
[z_free, alpha, lambda].
```

The equality rows enforce

$$Bz - 1 * \alpha = 0$$

for each positive-weight component.

The active-set logic removes materially negative free coefficients while ensuring that every constrained component retains at least one free variable. It can also re-enter fixed-zero variables that violate the dual conditions.

The solve is accepted only when the coefficients, common amplitude, equality residuals, and multiplier conditions are numerically consistent.

21. Active support

The expensive reduced solve contains only a subset of the full $C \times P$ columns. The outer solver maintains a boolean active mask and a scaled coefficient vector.

The important semantic rule is:

A proposed population is not judged by adding it to a frozen solution. The complete trial support is re-solved jointly.

This matters especially under the hard component-mass constraint because an improving direction normally requires coefficient mass to move away from one population while moving toward another.

A sparse active support is therefore not itself evidence of failure. The relevant diagnostics are KKT feasibility and whether broad joint-support trials can improve the objective.

22. KKT diagnostics

At each outer iteration the solver emits a `kkt` record containing quantities such as:

```
n_working_active
n_positive
n_zero_free
max_grad_active
max_grad_inactive
max_grad_dual
max_grad_data
max_grad_regularisation
max_grad_numerical_ridge
max_grad_constraint
kkt_violation
kkt_tol
active_ok
dual_ok
support_stationary
best_dual_col
best_dual_orbit
best_dual_population
ridge
alpha
constraint_dot_lambda
```

`active_ok` checks stationarity on positive active coefficients. `dual_ok` checks the relevant zero/inactive inequalities. `support_stationary` describes the current restricted support.

In the unconstrained pathway, satisfying the NNLS KKT conditions can terminate directly. In constrained mode, the solver can continue with joint-support exploration because a useful feasible redistribution can involve several columns simultaneously.

23. Joint-support exploration

The current solver replaced the older one-column promotion logic with joint-support exploration.

For each component it constructs up to three candidate families:

1. exchange candidates;
2. data-gradient candidates; and
3. coverage candidates.

Each family is assembled into a joint batch spanning as many components as possible, and the entire current support plus proposal is re-solved.

23.1 Exchange screening

For constrained components, the solver identifies currently active positive `donor` columns in the same component. Candidate exchange scores are based on the physical-space objective gradient relative to the best donor direction.

This attempts to screen for the operation the constrained solve actually needs to perform: redistribute mass within a component rather than merely add mass.

23.2 Data-gradient screening

A second family uses the raw data-fit gradient. It provides a complementary view that is not dominated by the current multiplier correction.

23.3 Rotating coverage

A third family deliberately samples inactive columns independent of the screening-gradient sign. This prevents highly correlated or multiplier-hidden populations from being permanently excluded.

The current constants are:

```

explore_top_per_orbit      = 6
explore_coverage_per_orbit = 3
explore_batch_size        = max(48, min(256, 4*C))
explore_batches_per_iter  = 3
explore_fail_patience     = 12

```

For $C=20$, the default joint batch cap is therefore 80.

24. Coarse-to-fine dyadic exploration

Sequential population offsets can be scientifically poor when adjacent SSP indices correspond to nearly identical spectra. The current code uses `_dyadic_offset()` for coarse-to-fine midpoint subdivision.

For a stride of 120, the intended sequence begins approximately as

```
0, 60, 30, 90, 15, 75, 45, 105, ...
```

rather than

```
0, 1, 2, 3, ...
```

The same dyadic rotation is also used to rotate the per-component candidate lists before a globally capped joint batch is assembled. This removes a systematic preference for low component indices.

24.1 Round-robin batch assembly

After rotating the component list, the solver selects candidate depth zero from each component, then depth one, and so on until `explore_batch_size` is reached.

This makes a capped batch broadly component-diverse instead of filling the batch with all candidates from early-index components.

25. Exploration acceptance

For every proposal, the solver forms

```
trial_active = current_active union proposal
```

and solves the complete reduced problem.

It computes both the data objective and the scientific regularization objective. A trial is accepted only when the total objective improves by more than a scale-aware floating-point tolerance:

```
64 * eps_float64 * max(1, abs(current_total_obj)).
```

Diagnostic `exploration_trial` records include:

```
proposal
proposal_n
trial_n_active
solve_accepted
accepted
current_data_obj
trial_data_obj
data_improvement
relative_data_improvement
current_regularisation_obj
trial_regularisation_obj
total_improvement
proposal_grad_data
proposal_grad_constraint
proposal_grad_total
proposal_grad_x
```

The re-solved trial objective is authoritative; the screening gradients are only candidate-generation diagnostics.

26. Checkpointing and FitTracker

When `tracker_mode != 'off'`, `PipelineRunner` creates a `FitTracker` sidecar writer process.

The tracker uses a multiprocessing queue and persists matching pairs of:

```
/Fit/x_last
/Fit.attrs['solver_state_json']
```

The iteration stored on `x_last` must match the iteration in the JSON state for a full resume to be accepted.

The active solver reads

```
CUBEFIT_SOLVER_CHECKPOINT_EVERY
```

with a default of 50. The current `kz_run.py` wrapper sets it to 1.

The tracker queue size is controlled by

```
CUBEFIT_TRACKER_QSIZE
```

with default 8192.

`FITTRACKER_START` selects the preferred tracker process start method; the tracker tries the preferred method and then available fallbacks.

27. Solver diagnostics and JSONL

The current constrained solver directly reads:

Variable	Default	Meaning
<code>CUBEFIT_DIAG_LEVEL</code>	1	diagnostic verbosity

Variable	Default	Meaning
CUBEFIT_DIAG_STRIDE	1	diagnostic stride
CUBEFIT_DIAG_TOPK	12	compact top-k summaries
CUBEFIT_DIAG_JSONL	unset	JSONL output path
CUBEFIT_SOLVER_CHECKPOINT_INTERVAL	50	checkpoint interval

A useful development configuration is:

```
export CUBEFIT_DIAG_LEVEL=2
export CUBEFIT_DIAG_STRIDE=1
export CUBEFIT_DIAG_TOPK=12
export CUBEFIT_DIAG_JSONL="$PWD/diagnostics.jsonl"
export CUBEFIT_SOLVER_CHECKPOINT_INTERVAL=1
```

27.1 Setup records

setup records establish the actual matrix dimensions, mask size, tile count, process count, and BLAS thread count.

mono_setup adds the active-set settings, regularization, constraint mode, canonical orbit shape, and exploration configuration.

Always inspect these before interpreting later diagnostics.

27.2 Plotting diagnostics

plotting.plot_diagnostic_jsonl_dashboard() can combine one or more JSONL files into a live/static solver dashboard.

28. Environment variables: current versus historical

The repository contains many CUBEFIT_* names because older solver pathways remain in the source tree.

28.1 Live in the current production path

The following are directly read by modules on the current path:

```
CUBEFIT_DIAG_LEVEL
CUBEFIT_DIAG_STRIDE
CUBEFIT_DIAG_TOPK
CUBEFIT_DIAG_JSONL
CUBEFIT_SOLVER_CHECKPOINT_INTERVAL
CUBEFIT_TRACKER_QSIZE
CUBEFIT_GEN_PROCESSES
CUBEFIT_GEN_BLAS_THREADS
CUBEFIT_RDCC_NBYTES
CUBEFIT_RDCC_NSLOTS
CUBEFIT_RDCC_W0
```

28.2 Historical or auxiliary for the current constrained solve

kz_run.py still sets variables such as

```
CUBEFIT_LAMBDA_WEIGHTS_ENABLE
CUBEFIT_GLOBAL_TAU
CUBEFIT_GLOBAL_ENERGY_BLEND
CUBEFIT_ZERO_COL_REL
CUBEFIT_LAMBDA_AGE
CUBEFIT_LAMBDA_ASMOOTH
CUBEFIT_LAMBDA_FLAT
CUBEFIT_LAMBDA_GROUP
```

```

CUBEFIT_LAMBDA_L1
CUBEFIT_LAMBDA_L2
CUBEFIT_MAX_INV_D
CUBEFIT_ZERO_COL_DATAFLOOR_MUL
CUBEFIT_ZERO_COL_ABS
CUBEFIT_ORBIT_PRIOR_WEIGHT

```

but `streaming_nnls_constrained.py` does not read these names in the current production solve. They should not be treated as controls for the v2.0 active solver unless another explicitly selected code path consumes them.

29. Reconstruction of `/ModelCube`

`loadCubeFit()` checks whether an existing `/ModelCube` is compatible with the current `X_global`. The status check can require float64 output and uses an `x_digest` stamp to avoid reusing a model reconstructed from a different solution.

If reconstruction is needed, the function chooses between

```

reconstruct_modelcube_fast
reconstruct_modelcube_fast_parallel

```

based on resolved parallelism.

The reconstruction contracts the stored (C, P) coefficient matrix against HyperCube spatial slabs without materializing a separate global two-dimensional design matrix.

After reconstruction, `loadCubeFit()` stamps `/ModelCube` with metadata including the coefficient digest, shape, arithmetic dtype, and generator.

30. `loadCubeFit()` diagnostics

`loadCubeFit()` computes several residual summaries from `/DataCube` and `/ModelCube` over the fitting mask.

The principal per-spatial-bin fit-quality metric is the median absolute symmetric fractional residual:

```

frac_resid = (data - model) / [0.5 * (data + model)]
Q_s        = 100 * median(abs(frac_resid)).

```

The function also computes:

- raw residual RMS;
- surface-brightness residuals divided by `BinCounts`;
- fractional RMS residual;
- fractional NMAD; and
- median fractional residual.

The best/worst spectrum plotting utilities use the resulting fit metric.

31. Orbit-constraint plot

`compare_orbit_vs_solution_absolute()` now diagnoses the constraint actually imposed by v2.0.

For

```

m_c = sum_p X[c,p]
alpha = mean(m_c over positive-weight components),

```

it plots:

1. normalized dynamical weights as context;
2. `m_c / alpha` with a target line at one; and

3. $(m_c - m_{\text{target}}, c) / \alpha$ as a residual bar chart.

The dynamical weights are not plotted as coefficient-mass targets because those weights are already encoded in the model-normalized HyperCube.

32. Validation and exact closure tests

The validation workflow is now separated into `validation.py`.

32.1 Create a minimal mock-data file

```
from CubeFit.validation import makeValidationCube

makeValidationCube(
    'CubeFit/NGC4365/hypercube_3_01.h5',
    'CubeFit/NGC4365/hypercube_3_01_mock-data.h5',
)
```

The source file must contain `/ModelCube` and `/ObsPix`.

The output is a new HDF5 file containing only:

```
/DataCube
/ObsPix
```

plus validation provenance attributes on `/DataCube`.

This is intentionally much smaller than the source HyperCube. Copying the entire source HDF5 and deleting `/HyperCube` is not equivalent because HDF5 may retain freed space in the physical file until it is repacked.

32.2 Load validation data in `genCubeFit()`

`loadValidationCube()` requires the synthetic file's `/ObsPix` to match the current `_oneTimeSpec` grid to tight tolerance. It requires the synthetic cube shape to be `(nSpat, nLSpec)` and returns a contiguous `(nLSpec, nSpat)` array for `genCubeFit()`.

Example:

```
genCubeFit(
    ...,
    runSwitch='fit',
    validationPath='CubeFit/NGC4365/hypercube_3_01_mock-data.h5',
    validationDataset='/DataCube',
    validationTag='closure-01',
)
```

`validationTag` is not the input filename. It is the suffix for the new fit and its validation figure directory.

32.3 Validation output separation

For validation generation, `genCubeFit()` writes figures under

```
<galaxy>/validation/<validationTag>/
```

rather than the production `<galaxy>/figures/` directory.

`loadCubeFit()` uses the same validation figure directory when either `validationPath` or `validationTag` marks the analysis as validation. In practice the tagged output HyperCube is selected by `validationTag`.

32.4 Compare recovered coefficients with truth

```
from CubeFit.validation import compare_validation_solution

out = compare_validation_solution(
    'CubeFit/NGC4365/hypercube_3_01.h5',
    'CubeFit/NGC4365/hypercube_3_01_closure-01.h5',
    save_path='CubeFit/NGC4365/validation/closure-01/x_recovery.png',
)
```

The function automatically locates the sibling `x_*.h5` files and compares `/X_global` coefficient by coefficient. It reports relative L1, L2 and Linf errors, correlation, cosine similarity, population-summed recovery, and component coefficient-mass recovery.

32.5 What a closure failure means

A validation failure can originate from several distinct layers:

1. the mock data and fit do not use identical spectral grids;
2. the HyperCube was rebuilt with different normalization or convolution settings;
3. the solver objective/constraint differs from the generating fit;
4. the active search did not recover an equivalent support; or
5. the spectra are reproduced but x is non-identifiable because SSP columns are highly correlated.

Always inspect both spectral closure and coefficient closure. Excellent spectral closure does not guarantee one-to-one recovery of a non-unique SSP coefficient vector.

33. Storage management

`/HyperCube/models` normally dominates disk use:

```
uncompressed bytes ~= S * C * P * L * 4
```

because the stored dtype is float32.

The physical solution is tiny by comparison:

```
bytes(X_global) ~= C * P * 8
```

for float64 coefficients.

33.1 Archive and restore

`hdf5_manager.py` provides:

```
archive_hypercube_models(...)
restore_hypercube_models(...)
classify_hypercube_models(...)
```

Archiving rewrites `/HyperCube/models` with gzip level 9 plus shuffle on a work copy, validates the result, stamps storage state, and only then replaces the original file.

Restore performs the inverse solver-ready rewrite to an uncompressed validated layout.

These operations use work copies to avoid replacing the original file until validation succeeds.

34. Current solution format

The current production solver returns a two-dimensional float64 matrix

```
X_global.shape == (C,P).
```

`PipelineRunner` writes this directly to `/X_global` and sets

```
layout = C_P
P       = <population count>.
```

`genCubeFit()` writes the same two-dimensional layout into the sibling solution HDF5 file.

Historical documentation that describes `/X_global` as a flattened $(C \times P)$ vector is stale for the current write path, although several internal solver functions still flatten temporarily with row-major `order='C'`.

35. Debugging utilities

`cube_debug.py` contains targeted audits used during solver development.

Particularly relevant functions include:

```
debug_test_projectors_on_h5
test_solution_rmse
diagnose_orbit_weight_encoding
compare_sidecar_solution_to_hypercube
check_c3_cfine_additivity
test_c3_copy_in_cfine
```

`diagnose_orbit_weight_encoding()` is useful for separating three quantities:

- input dynamical component weights;
- LOSVD/HyperCube amplitudes; and
- fitted coefficient mass.

This distinction was central to identifying the double-application problem in the old orbit constraint.

36. Reference and audit solvers

`streaming_nnls_constrained.py` also contains dense/reference functions:

```
monolithic_nnls_scipy
monolithicNNLS
monolithicSolver
```

They are useful for small problems and mathematical audits but are not the production path selected by `PipelineRunner`.

The repository history explicitly notes that fully monolithic approaches become expensive as `C` grows. Use them as reference checks, not as the default large-production method.

37. Performance interpretation

37.1 HyperCube build

The dominant costs are FFT convolution, template-to-observed matrix multiplication, memory traffic, HDF5 writes, and process scheduling.

In this snapshot, only one slab future is outstanding at a time, so increasing `processes` does not increase concurrent tile computation. BLAS threading and single-worker efficiency therefore remain particularly important.

37.2 Solver

The solver's large-data memory advantage comes from streaming spatial slabs instead of constructing the full global design matrix or a full $CP \times CP$ normal matrix.

The expensive operations are:

- full streamed gradient / reduced-product passes;
- dense reduced active-set solves; and
- repeated joint-support trials.

Increasing the candidate list itself is cheap in memory. The cost comes from the size and number of reduced trial solves.

37.3 Sparse support

A support of only a few dozen coefficients can be mathematically legitimate in a highly correlated SSP library. Do not use support size as a convergence criterion.

Audit instead:

```

KKT violation
inactive dual gradient
joint-trial objective improvement
coverage rounds
explore_fail_count
validation closure

```

38. Troubleshooting

38.1 ... does not contain /ModelCube

If the path is a minimal mock-data file created by `makeValidationCube()`, its synthetic spectra are intentionally stored as `/DataCube`, not `/ModelCube`. Pass

```
validationDataset='/DataCube'
```

to `genCubeFit()`.

38.2 Validation output file does not exist

`validationTag` names the new fit, not the input mock-data file. If the input is `hypercube_3_01_mock-data.h5` and the tag is `lambda_1e-4`, the analysis expects the new fit file `hypercube_3_01_lambda_1e-4.h5`.

38.3 Mock-data file is tens of GB

A minimal validation input should contain only `/DataCube` and `/ObsPix` and should be orders of magnitude smaller than the full HyperCube. Recreate it with `makeValidationCube()` rather than copying the source HDF5.

38.4 Constrained fit is much worse than unconstrained fit

Check, in this order:

1. `/HyperCube.attrs['norm.mode']`;
2. whether the component amplitudes are already encoded in `A`;
3. `mono_setup.orbit_constraint_mode`;
4. whether a stale proportional-orbit-weight solver/checkpoint is being used;
5. the constrained KKT diagnostics;
6. joint-support trial improvements; and
7. a synthetic closure test.

For the current production model-normalized path, the expected constraint mode is `equal_component_mass`.

38.5 Full resume rejected

A full checkpoint is intentionally rejected when regularization, constrained versus unconstrained mode, current constraint schema, or stored orbit weights are incompatible. Use `warm='saved_x'` to reuse only the physical solution.

38.6 More CPUs are slower

First verify what is actually parallel. In the supplied HyperCube builder only one tile future is active at a time. Also inspect BLAS threading and the Slurm `cpuset`. More worker processes are not automatically more throughput.

38.7 Orbit plot target looks wrong

For the current constraint, the coefficient-space target is

```
m_c / alpha = 1
```

for every positive-weight component. The normalized dynamical weights belong in a separate context panel, not on the coefficient-mass target line.

38.8 /X_global shape assumptions fail

Current production output is (C, P) . Code that assumes a stored flat $(C \times P,)$ array should reshape conditionally rather than requiring the old layout.

39. Recommended production checklist

Before a long run:

1. confirm the intended C , P , S , and wavelength range;
2. verify the mask convention: `True` means keep;
3. run the convolution/rebin preflight;
4. inspect the estimated velocity bias if enabled;
5. confirm `norm.mode='model'` for the current orbit-constraint semantics;
6. decide explicitly whether `orbitWeights` should be enabled;
7. use `regularisation_scale=0.0` for strict closure/reference tests;
8. set BLAS/OpenMP variables before Python starts;
9. inspect the resolved process/thread pair;
10. enable JSONL diagnostics for solver-development runs;
11. use `saved_x` after changing solver mathematics; and
12. use a unique `validationTag` for validation outputs.

After the run:

1. inspect the final stop category and KKT diagnostics;
2. reconstruct `/ModelCube` and verify its `x_digest`;
3. inspect spatial and spectral residuals;
4. inspect the orbit-constraint plot;
5. inspect active support without assuming that dense support is required;
6. run one-to-one validation when changing solver mathematics; and
7. archive diagnostics with the solution.

40. Source-derived API index

The signatures below are taken from the supplied repository snapshot.

40.1 `kz_fitSpec.py`

```
genCubeFit(galaxy, mPath, decDir=None, nCuts=None, proj='i', SN=90,
           full=False, slope=1.3, IMF='KB', iso='pad', weighting='luminosity',
           lOrder=4, rescale=False, specRange=None, lsf=False, band='r', smask=None,
           method='fsf', varIMF=False, source='ppxf', redraw=False,
           runSwitch='gen', bias=True, **kwargs)

loadCubeFit(galaxy, mPath, decDir=None, nCuts=None, proj='i', SN=90,
            full=False, slope=1.3, IMF='KB', iso='pad', weighting='luminosity',
            lOrder=4, rescale=False, specRange=None, lsf=False, band='r', smask=None,
            method='fsf', varIMF=False, source='ppxf', redraw=False,
            pplots=['sfh', 'spec', 'mw', 'proj'], **kwargs)

reconstruct_modelcube_fast(...)
reconstruct_modelcube_fast_parallel(...)
modelcube_status(...)
parallel_spectrum_plots(...)
plot_best_worst_spectrum_fits_stacked(...)
compare_orbit_vs_solution_absolute(...)
```

40.2 hypercube_builder.py

```
preflight_hypercube_convolution(...)
estimate_global_velocity_bias_prebuild(...)
build_hypercube(...)
ensure_global_column_energy(...)
read_global_column_energy(...)
convert_hypercube_norm(...)
assert_preflight_ok(...)
```

40.3 hdf5_manager.py

```
open_h5(...)
H5Manager
invalidate_done(...)
print_hypercube_done_status(...)
archive_hypercube_models(...)
restore_hypercube_models(...)
classify_hypercube_models(...)
read_observed_spectrum(...)
read_template_spectrum(...)
read_losvd(...)
read_model_basis(...)
reconstruct_model_spectrum(...)
plot_prefit_panel(...)
live_prefit_snapshot_from_models(...)
```

40.4 pipeline_runner.py

```
PipelineRunner(h5_path)
PipelineRunner.build_hypercube(**kwargs)
PipelineRunner.solve_all_mp_batched(...)
```

40.5 streaming_nnls_constrained.py

```
MPCConfig
streamActiveSetNNLS(...)
solve_streaming_nnls(...)
monolithic_nnls_scipy(...)
monolithicNNLS(...)
monolithicSolver(...)
```

40.6 validation.py

```
makeValidationCube(source_path, output_path)
loadValidationCube(validationPath, spLL, nSpat, dataset='/DataCube')
compare_validation_solution(input_path, validation_path, save_path=None)
```

40.7 plotting.py

```
plot_aperture_fit(...)
plot_white_light_images(...)
plot_model_decomposition(...)
plot_diagnostic_jsonl_dashboard(...)
```

40.8 cube_utils.py

Important current/diagnostic utilities include:

```
resolve_parallelism(...)
cpuset_count()
blas_threads_ctx(...)
run_wavelength_checks(...)
upscaleSolution(...)
constrainUpscaledSolution(...)
diagnose_hypercube_scaling(...)
```

The module also contains older Kaczmarz-era softbox, lambda-weight, and support utilities. Their presence does not mean they are active in the current `PipelineRunner -> solve_streaming_nnls` pathway.

40.9 cube_debug.py

```
debug_test_projectors_on_h5(...)
nnls_seed_diagnostics(...)
compare_sidecar_solution_to_hypercube(...)
check_c3_cfine_additivity(...)
test_c3_copy_in_cfine(...)
test_solution_rmse(...)
diagnose_orbit_weight_encoding(...)
```

41. Building the repository documentation

The repository Makefile defines:

```
DOC  = docs/CubeFit.md
HTML = docs/CubeFit.html
PDF  = docs/CubeFit.pdf
```

and uses Pandoc for HTML and Pandoc + XeLaTeX for PDF.

The supplied `mkdocs.yml` points `docs_dir` at `docs` and lists `CubeFit.md` in the navigation. It also references `index.md`; the supplied Repomix snapshot does not list `docs/index.md`, so add one or update the MkDocs navigation if `mkdocs serve` requires it.

The generated HTML and PDF should be treated as build products. Edit `docs/CubeFit.md` and regenerate them rather than editing the rendered files by hand.

42. Snapshot-specific audit notes

The documentation overhaul intentionally records several discrepancies found between the old documentation/docstrings and the supplied implementation:

- the production solver is streaming active-set NNLS, not the old Kaczmarz pathway;
- current `/X_global` writes are two-dimensional (C, P) ;
- model-normalized HyperCube amplitudes make the old proportional orbit-coefficient target inappropriate;
- the current constraint is `equal_component_mass`;
- the current constrained solver does not consume the old lambda-weight environment knobs;
- the builder and solver worker initializers do not set `BLIS_NUM_THREADS`;
- the current HyperCube executor loop has only one outstanding slab future;
- `populate_from_arrays()` writes `/CompWeights`, despite a stale docstring mentioning `/OrbitWeights`; and
- `solve_streaming_nnls()` has a stale block-coordinate docstring even though the current production path calls `streamActiveSetNNLS()`.

These notes are included so future documentation changes do not accidentally reintroduce behavior from historical solver generations.

43. Maintenance rule

Changes to any of the following should be treated as documentation-breaking changes and updated in the manual in the same revision:

- HyperCube normalization;
- orbit-constraint equations;
- physical/scaled coefficient definitions;
- active-set exploration;

- checkpoint compatibility;
- validation-file schema;
- /X_global layout;
- solver objective/regularization;
- multiprocessing semantics; and
- diagnostic-field meaning.

For CubeFit these are scientific semantics, not merely implementation details.